WATCHMAN

Local Windows File Integrity Monitoring & Audit System

Comprehensive User Guide & Technical Operations Manual

Software Version:	0.1.0
Target Platform:	Microsoft Windows 10 & 11 (64-bit)
Security Model:	User-Space execution (No Administrator privileges required)
Document Release:	Production User Manual
Developed By:	Avanish Kumar

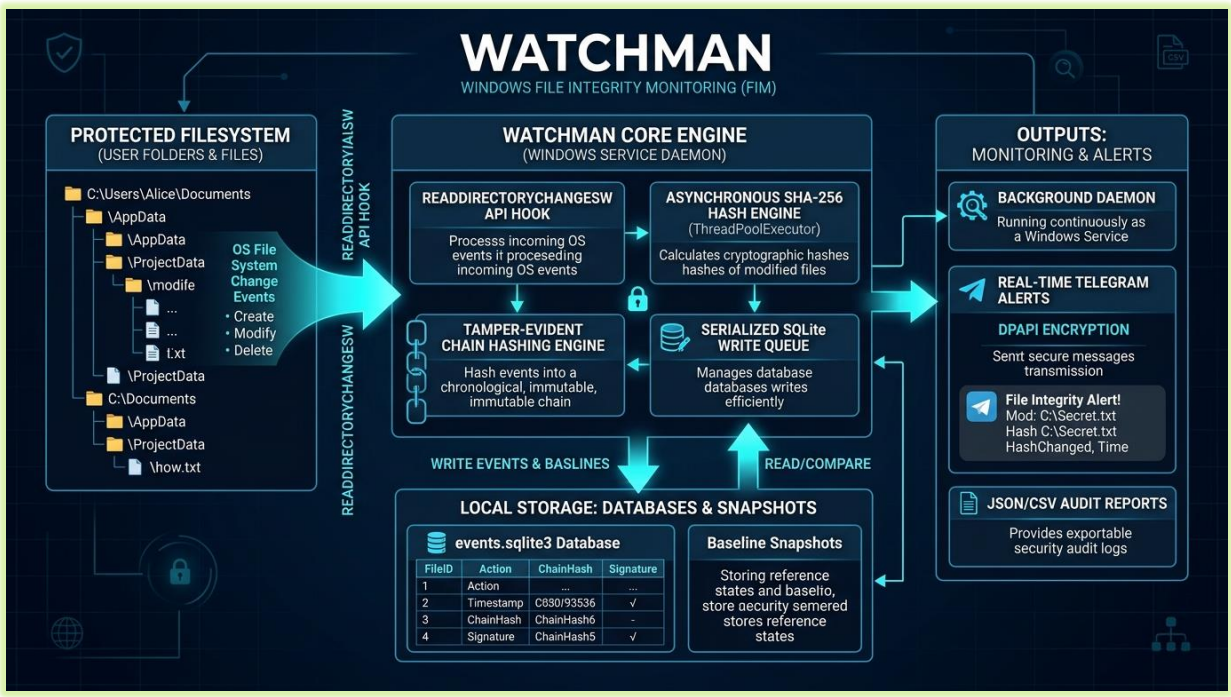
---

1. Executive Summary & Architecture Overview

Watchman is a purpose-built, user-controlled file integrity monitoring (FIM) and change auditing utility for Microsoft Windows. Unlike enterprise security agents that install hidden system-level services, consume background resources, or transmit private data to remote SaaS dashboards, Watchman is designed strictly around user autonomy, transparency, and data ownership.

Key Architecture Highlights:

- **Zero Administrative Privileges Required:** Runs entirely within standard user rights. It never touches system registry roots, never modifies Windows kernel drivers, and does not require UAC elevation.
- **Multi-Deployment Isolation:** Define completely isolated monitoring setups (e.g., 'work', 'finance', 'sourcecode'), each with independent directory sets, exclude rules, and local event databases.
- **Tamper-Evident SHA-256 Chain Hashing:** Every recorded filesystem event stores a cryptographic SHA-256 hash that chains directly into the previous record. Sequential deletions or row modifications are immediately detectable.
- **Non-Blocking Concurrent Operations:** Disk I/O and file hashing are handled through dedicated background threads (ThreadPoolExecutor and a dedicated SQLite worker thread), guaranteeing real-time responsiveness without locking.
- **Baseline Snapshot & Differential Auditing:** Take instant point-in-time cryptographic snapshots of entire directory trees, and verify files at any time to catch modified, missing, or unauthorized new files.
- **Optional Encrypted Real-Time Alerts:** Configurable Telegram bot alerts with credentials secured inside Windows Credential Manager via native DPAPI encryption.



2. System Requirements & Prerequisites

Before installing Watchman, ensure your Windows workstation meets the following prerequisites:

Component	Minimum Requirement	Recommended
Operating System	Windows 10 (Build 19041+) or Windows 11	Windows 11 (64-bit)
Python Runtime	Python 3.10+	Python 3.12 or 3.13 (64-bit)
Shell Environment	Windows PowerShell 5.1	PowerShell 7 (pwsh) or Windows Terminal
User Permissions	Standard User Account	Standard User Account (No Admin needed)

> **[IMPORTANT WARNING]** When installing Python from python.org, you MUST select the checkbox labeled 'Add python.exe to PATH' during setup to ensure terminal accessibility.

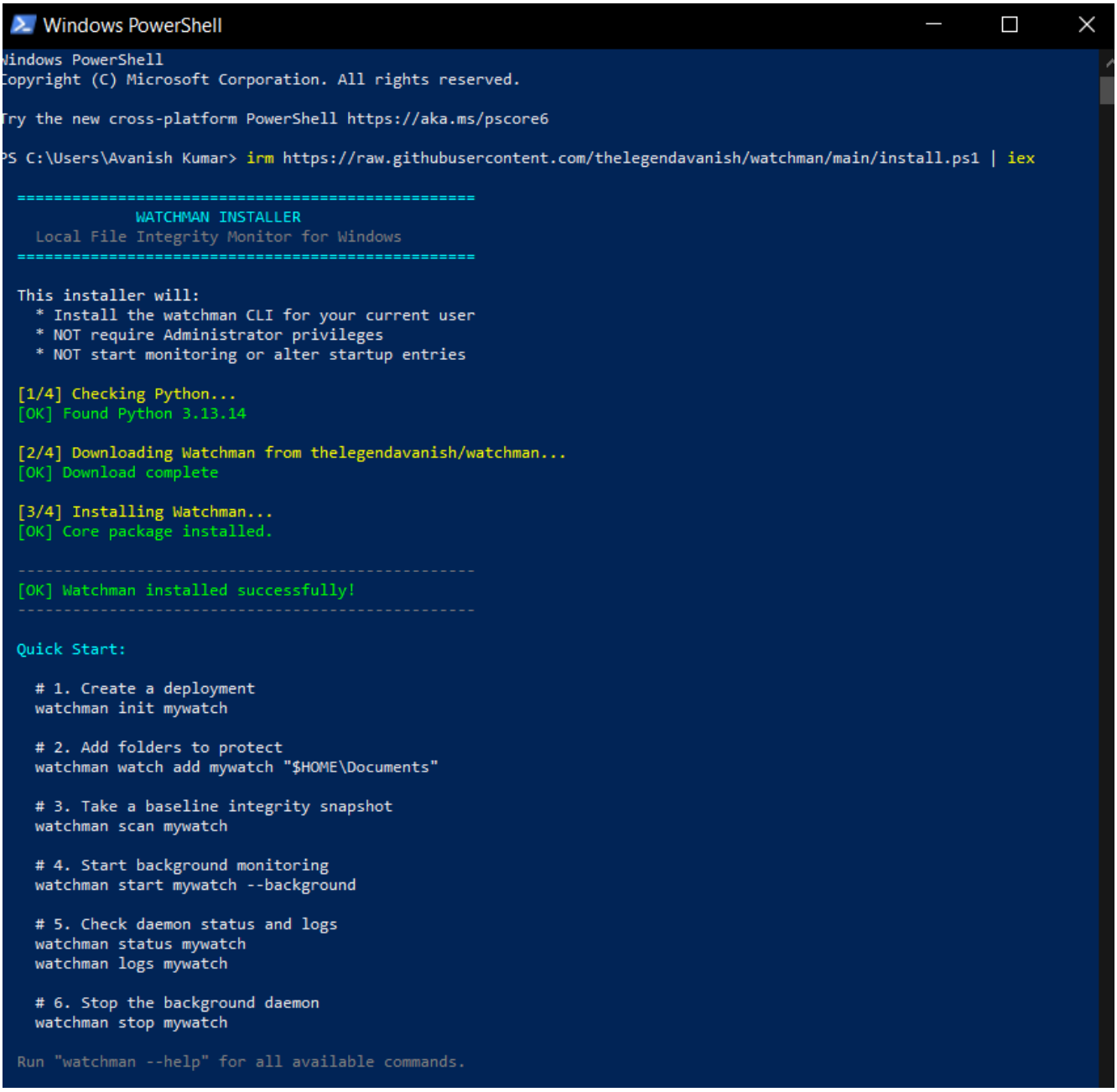
3. Installation Guide

3.1 Method A: Remote Terminal Installer (Recommended)

The fastest and cleanest way to install Watchman is using PowerShell's native Invoke-RestMethod ('irm | iex') pipeline. Open a PowerShell terminal as your normal user and execute:

```
irm https://raw.githubusercontent.com/thelegendavanish/watchman/main/install.ps1 | iex
```

The remote installer automatically verifies Python version compatibility, downloads the official repository release, installs Watchman into your user site-packages via pip, and registers the 'watchman' binary onto your PATH.



3.2 Method B: Manual Offline Installation

If you are working in an air-gapped or offline environment, clone or extract the repository and run:

```
cd C:\path\to\watchman
python -m pip install --user .

# Optional: Windows Credential Manager support
python -m pip install --user ".[secure]"
```

3.3 Verifying the Installation

Verify that Watchman is properly recognized by opening a fresh PowerShell session and executing:

```
watchman --help
```

4. Configuring Deployments & Watched Directories

Watchman organizes all monitoring targets into independent named 'deployments'. Each deployment stores its configuration, baseline snapshots, event logs, and daemon state in an isolated directory located under:

```
%LOCALAPPDATA%\Watchman\deployments\<deployment-name>\
    config.json       <- Protected paths, exclude patterns, startup flags
    events.sqlite3    <- Tamper-evident chain-hashed SQLite event log
    monitor.pid       <- Active daemon PID tracking file
    monitor.log       <- Daemon standard output & error log
```

4.1 Initializing a Deployment

To create a new deployment called 'secure-docs':

```
watchman init secure-docs
```

> **[NOTE]** Initializing a deployment only creates the database schema and configuration file. It does NOT start monitoring and does NOT alter Windows startup.

4.2 Adding Folders to Protect

Add one or more directories to be continuously watched for file modifications, creations, deletions, or moves:

```
watchman watch add secure-docs "C:\Users\YourName\Documents"
watchman watch add secure-docs "D:\CriticalProjects"
```

4.3 Applying File Exclusion Patterns

To filter out noisy temporary files generated by office suites or text editors, pass repeatable '--exclude' glob patterns:

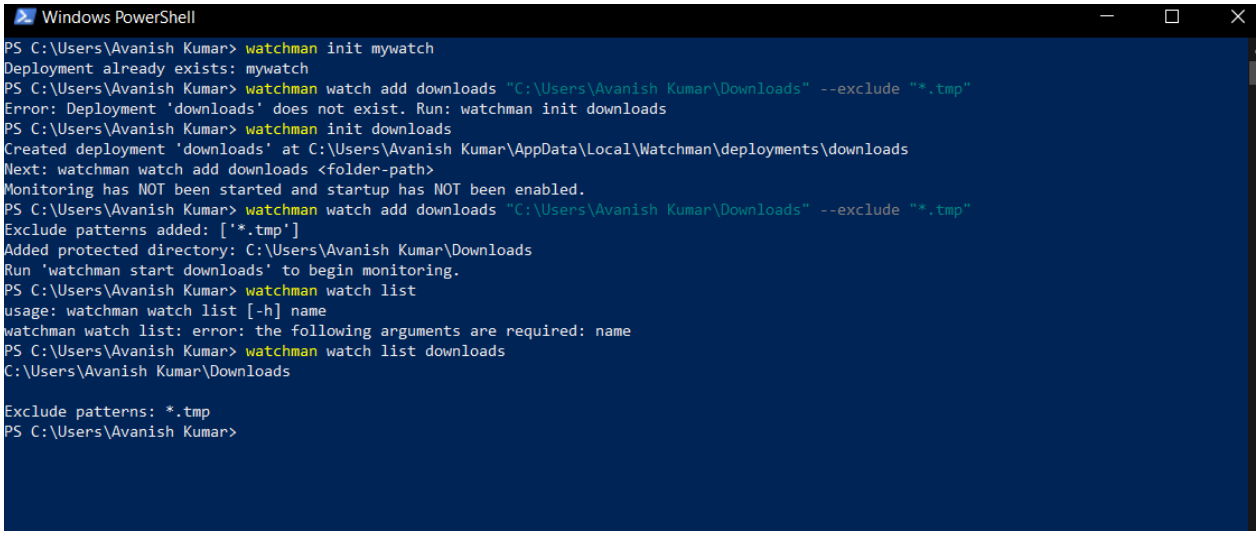
```
watchman watch add secure-docs "D:\SourceCode" --exclude "*.tmp" --exclude "~$*" --exclude "*.log"
```

4.4 Listing and Removing Watched Paths

Inspect all active directories and exclusion rules configured for a deployment:

```
watchman watch list secure-docs

# To remove a watched path:
watchman watch remove secure-docs "D:\CriticalProjects"
```



5. Baseline Cryptographic Integrity Auditing

In addition to real-time event streaming, Watchman provides an offline integrity engine. A baseline scan takes an immediate SHA-256 fingerprint snapshot of every file inside your protected directories. You can run an integrity verification at any point in time to detect unauthorized tampering.

5.1 Taking a Baseline Snapshot

```
watchman scan secure-docs
```

Watchman recursively crawls every watched directory, computes SHA-256 digests, and stores them in the local 'baselines' table.

5.2 Verifying Integrity Against Baseline

Run verification periodically or via scheduled tasks:

```
watchman verify secure-docs
```

If all files match the baseline, Watchman returns exit code 0:

```
Verifying integrity for deployment 'secure-docs'...
[OK] All files match the baseline. No changes detected.
```

If discrepancies exist, Watchman lists exact findings and exits with code 1:

```
[WARNING] 3 discrepancy(ies) found:

  MODIFIED  C:\Users\YourName\Documents\payroll.xlsx
        expected : 3a51f8b76e...
        actual   : 9f12d4a8bc...
  MISSING   C:\Users\YourName\Documents\secret.key
  NEW       C:\Users\YourName\Documents\malware.exe
        sha256   : e3b0c44298...
```

```
PS C:\Users\Avanish Kumar> watchman scan mywatch
Scanning all files under protected paths for 'mywatch'...
Baseline snapshot complete: 17 file(s) indexed.
PS C:\Users\Avanish Kumar> watchman verify mywatch
Verifying integrity for deployment 'mywatch'...
[OK] All files match the baseline. No changes detected.
PS C:\Users\Avanish Kumar> watchman verify mywatch
Verifying integrity for deployment 'mywatch'...
[WARNING] 1 discrepancy(ies) found:

  NEW       C:\Users\Avanish Kumar\Documents\Assets\New Text Document.txt
            sha256   : e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
PS C:\Users\Avanish Kumar>
```

6. Live File Monitoring & Process Management

6.1 Foreground Monitoring Mode

To monitor events interactively in your current console (ideal for testing):

```
watchman start secure-docs
```

Watchman prints real-time events as they occur. Press Ctrl+C at any time to safely shut down the observer.

6.2 Background Daemon Mode

To detach Watchman and keep it running silently in the background while freeing your terminal:

```
watchman start secure-docs --background
```

Watchman launches a detached Windows process group via kernel32 APIs, writes the process ID to 'monitor.pid', and redirects all daemon output to 'monitor.log'.

6.3 Checking Daemon Status

```
watchman status secure-docs

# Output:
# Deployment 'secure-docs': RUNNING (PID 14220)
# Log file: C:\Users\...\monitor.log
```

You can also view all deployments at once:

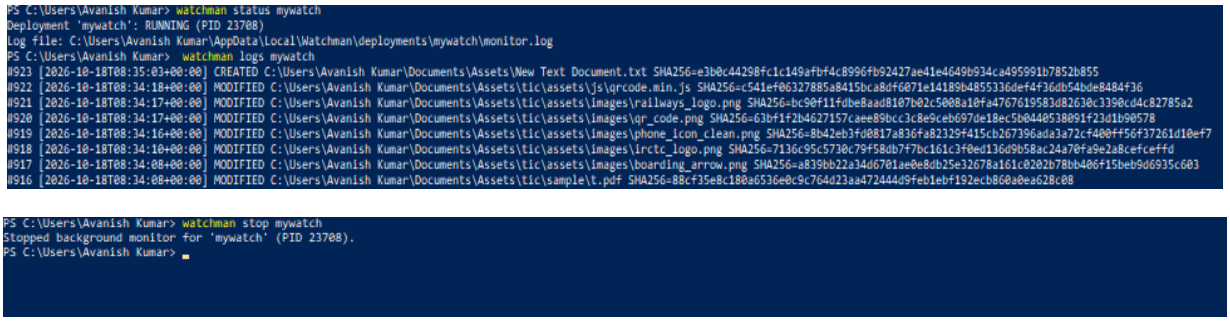
```
watchman list

# Output:
# secure-docs: 2 protected path(s); startup=disabled [RUNNING]
# finance: 1 protected path(s); startup=disabled
```

6.4 Stopping the Background Daemon

To gracefully terminate a running background daemon:

```
watchman stop secure-docs
```



7. Event Audit Logs & Data Export

7.1 Viewing Live Event History

```
watchman logs secure-docs --limit 30
```

Displays the most recent events in reverse chronological order, including timestamp, action, file path, and SHA-256 hash.

7.2 Exporting Audit Logs to JSON or CSV

Export the complete tamper-evident audit trail for reporting or ingestion into SIEM tools:

```
# Export to JSON
watchman export secure-docs --format json --output audit_report.json

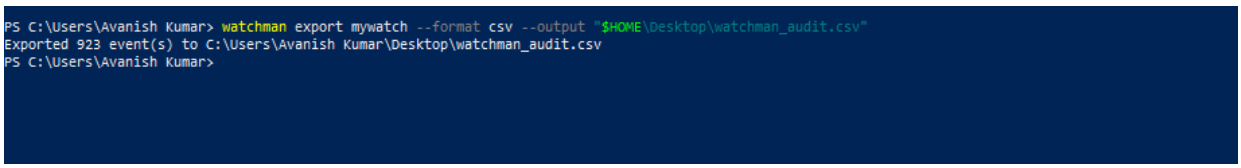
# Export to CSV (compatible with Excel & PowerBI)
watchman export secure-docs --format csv --output audit_report.csv
```

7.3 Clearing Event Logs

To purge the event table (with interactive safety prompt):

```
watchman clear secure-docs

# Or bypass prompt in automated scripts:
watchman clear secure-docs --yes
```



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	id	timestamp	event_type	path	destination	sha256	details	prev_event_hash						
2														
3	1	2026-10-1	modified	C:\Users\Avanish Kun		faffb987e(	file							
4														
5	2	2026-10-1	modified	C:\Users\Avanish Kun		204f1bcfb(	file	eb2f3fdc95e543c5109f15e0f75b3f2848f7616a40c7b88d5f0650d6fd993735						
6														
7	3	2026-10-1	modified	C:\Users\Avanish Kun		e6fb6ae17	file	86939c1cc03726c200ec30eec8902ab5908ca05ad5efe5dce26e4683b97b93a8						
8														
9	4	2026-10-1	deleted	C:\Users\Avanish Kumar\Docum			file	4c600ab05a01d1611f34ca7d5980fb6f507cb9d059353629660810326720 9afd						
10														
11	5	2026-10-1	modified	C:\Users\Avanish Kun		9d9674bc7	file	9ab9089073066ba3eae2c4638d747a9215ce400ec2604e2427e3b55c2a6ffb9a						
12														
13	6	2026-10-1	modified	C:\Users\Avanish Kun		6f3fbeb61	file	0870c0d0ae664fcbcbb4f415706cb3672a222573565835e709c787a92b9ec059						
14														
15	7	2026-10-1	modified	C:\Users\Avanish Kun		be1851b1f	file	d1ee8cad5ed919d25dfd7422199d2f57c542a413b5845bd14f27edc34b2cc1b2						
16														
17	8	2026-10-1	modified	C:\Users\Avanish Kun		0cad42b11	file	453d379386ddf524a87c81538a5e892699f899cd9f0e4d8bf291cb444f8c5794						
18														
19	9	2026-10-1	modified	C:\Users\Avanish Kun		2113231c3	file	4dafdef72f1484bacf480f187d4c8757c34e5b5ed770b8431871e61e31889583						
20														
21	10	2026-10-1	modified	C:\Users\Avanish Kun		23759e3f6	file	4ff78f0873999e77caea60b300a349ea587d51b46cda97a738d00b2832914866						
22														
23	11	2026-10-1	modified	C:\Users\Avanish Kun		261bea60(	file	c94933dec5bbe88696a9c9a7635c692e803ece44e78f626807821d3056c6e9f1						
24														
25	12	2026-10-1	modified	C:\Users\Avanish Kun		2cae6102f	file	62a917aa5f361d369de613026a0be5edd5e0cba2bd1e8cd69266551831aba05f						

8. Real-Time Telegram Security Alerts

Watchman can dispatch instant mobile alerts over Telegram whenever a critical file event occurs. Credentials are cryptographically stored in Windows Credential Manager using Windows DPAPI.

8.1 Configuring Telegram Alerts

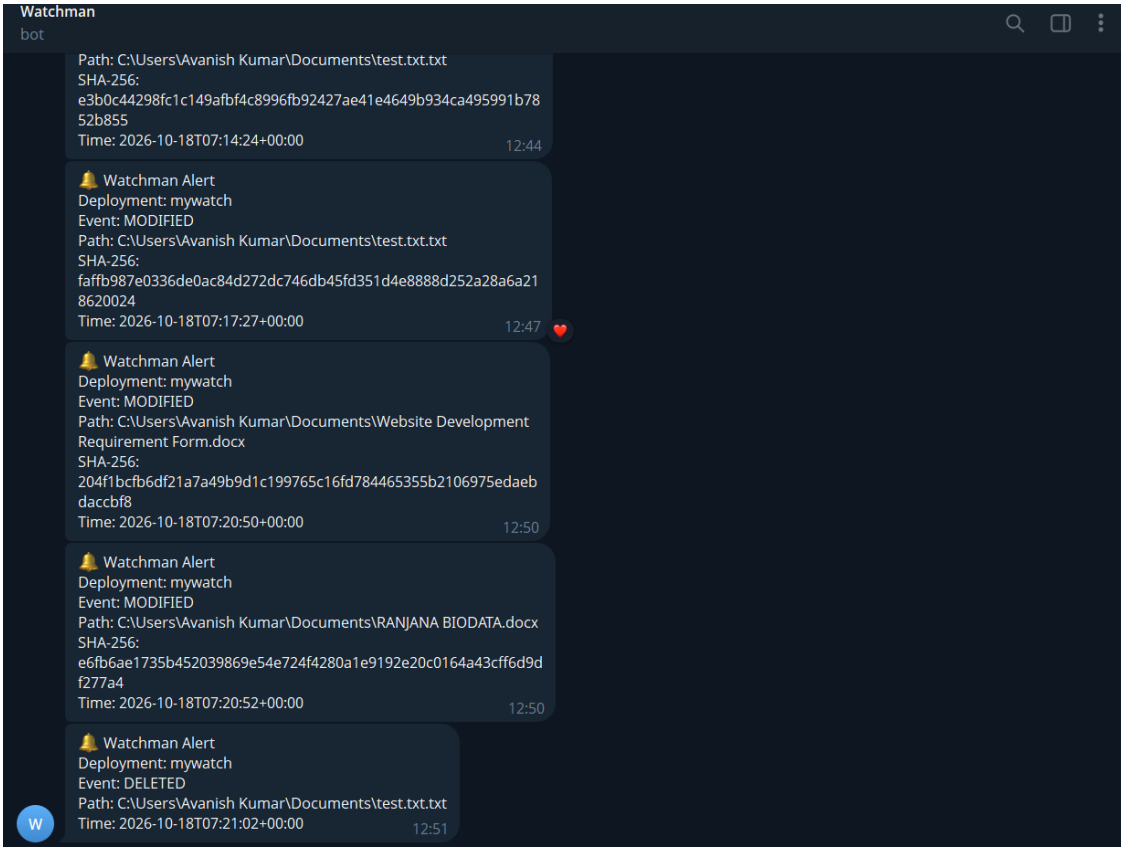
Obtain a bot token from @BotFather on Telegram, get your user or group chat ID, and run:

```
watchman telegram secure-docs --token "123456789:ABCdefGHIjklMNO" --chat-id "987654321"
```

```
PS C:\Users\Avanish Kumar> watchman telegram mywatch --token "123456:ABCdef..." --chat-id "YOUR_CHAT_ID"
Telegram settings saved in config file (install 'keyring' for secure storage).
Keep your bot token private.
PS C:\Users\Avanish Kumar>
```

8.2 Removing Stored Telegram Credentials

```
watchman telegram-clear secure-docs
```



9. Automatic Windows Logon Startup (Opt-In)

Watchman avoids hidden persistence mechanisms. If you choose to enable automatic monitoring at user logon, Watchman places a visible, user-accessible '.cmd' shortcut inside your standard Windows Startup directory:

```
%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\Watchman-<deployment>.cmd
```

9.1 Startup Commands

```
# Enable monitoring on logon
watchman startup enable secure-docs

# Check current startup status
watchman startup status secure-docs

# Disable logon startup
watchman startup disable secure-docs
```

10. Complete System Uninstallation

Watchman respects clean software uninstallation. When uninstalled, it leaves zero orphan files, services, or registry keys.

10.1 Automated One-Liner Removal

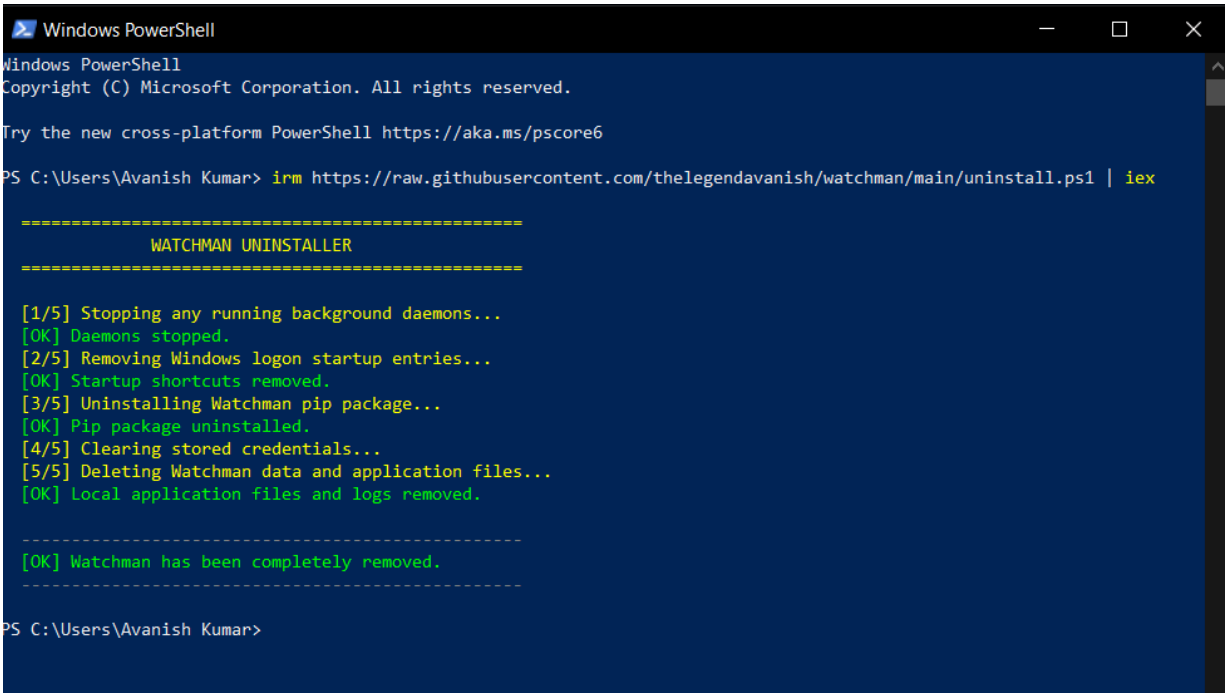
```
irm https://raw.githubusercontent.com/thelegendavanish/watchman/main/uninstall.ps1 | iex
```

10.2 Manual Removal Steps

```
# 1. Uninstall pip package
python -m pip uninstall -y watchman

# 2. Delete configuration and database files
Remove-Item -Recurse -Force "$env:LOCALAPPDATA\Watchman"

# 3. Clean startup shortcuts
Remove-Item -Force "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\Watchman-*.cmd"
```



11. Complete Command Reference Cheat Sheet

Command Syntax	Category	Description
watchman init <name>	Management	Initialize a new isolated deployment.
watchman list	Management	List all deployments and active daemon status badges.

watchman watch add <name> <dir>	Configuration	Add a directory to the protected monitoring list.
watchman watch add ... --exclude <pat>	Configuration	Exclude specific file glob patterns (e.g. *.tmp).
watchman watch remove <name> <dir>	Configuration	Remove a directory from the protected list.
watchman watch list <name>	Configuration	Display all protected paths and exclude rules.
watchman scan <name>	Integrity	Take an offline SHA-256 baseline snapshot of all files.
watchman verify <name>	Integrity	Compare current filesystem against baseline snapshot.
watchman start <name>	Execution	Start real-time monitoring in the foreground.
watchman start <name> --background	Execution	Spawn detached background daemon process.
watchman status <name>	Execution	Check if background daemon is active and print PID.
watchman stop <name>	Execution	Gracefully terminate active background daemon.
watchman logs <name> [--limit N]	Auditing	View recent recorded events with timestamps & hashes.
watchman export <name> --format json|csv	Auditing	Export complete tamper-evident audit trail.
watchman clear <name> [--yes]	Auditing	Permanently delete recorded events.
watchman telegram <name> --token --chat-id	Alerting	Store encrypted Telegram bot credentials.

12. Troubleshooting & Frequently Asked Questions

Q: Does Watchman consume heavy CPU while monitoring large folders?

A: No. Watchman relies on the native Windows ReadDirectoryChangesW API via the Python 'watchdog' library. It uses zero polling overhead. File hashing is dispatched asynchronously into a worker thread pool (max 4 threads), meaning your system remains fast and responsive.

Q: What happens if a file is modified while the system is turned off?

A: Real-time event hooks only catch events that occur while the monitor is active. For offline changes, run 'watchman verify <name>' periodically. The baseline verification checks every file against the original cryptographic snapshot and immediately catches offline changes.

Q: Where are my Telegram credentials saved?

A: When the optional 'keyring' module is installed, tokens are stored inside Windows Credential Manager under the service name 'Watchman'. They are encrypted using your Windows user DPAPI master key and never saved in plain text.

Q: What does 'prev_event_hash' mean in the database?

A: To prevent adversaries from secretly deleting audit rows from the SQLite database, each event incorporates the SHA-256 hash of the immediately preceding event row. If any historical row is altered or deleted, the cryptographic hash chain breaks, proving evidence tampering.